Reviewer Pack — Matousek Det-LB Submatrix Dispersion Lower-Bounds Friedman S...

Entry 2c4a5e3ab436 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-18 22:57:50 UTC

Matousek Det-LB Submatrix Dispersion Lower-Bounds Friedman Sign Rigidity

Entry ID: `2c4a5e3ab436` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-18 22:57:50 UTC

1. Conjecture

Field A (mathematical branch): Geometric / combinatorial discrepancy theory — Matousek 2013 determinant lower bound det-LB for hereditary discrepancy of set systems (Matousek 2013 'The determinant bound for discrepancy is almost tight', Israel J. Math.; Barany-Grinberg 1981; Lovasz-Spencer-Vesztergombi 1986 on LP-based discrepancy lower bounds; Chazelle 'The Discrepancy Method' 2000). The functional D_r(M) := max{|det(A)|^{2/r} : A is an r-by-r submatrix of M} uses max + |.| + integer determinants on submatrix families; the OUTER max over r-subsets is sup-style (no F_q polynomial extension preserves max-of-determinants under polynomial ring lifts), so the bridge is Aaronson-Wigderson algebrization-safe. An arXiv search 'Matousek determinant lower bound' AND ('matrix rigidity' OR 'Valiant rigidity' OR 'Friedman spectral') returns 0 direct hits and <5 adjacent papers (Lovasz-Spencer-Vesztergombi LP discrepancy on combinatorial incidence matrices, never on sign-matrix spectral rigidity). Distinct from blacklisted Bourgain-Tzafriri subcolumn dispersion (random column-subset operator norms — a Schatten ratio, not a determinant max), Halasz L^2 spectrum discrepancy (Laplacian-spectrum CDF deviation, not submatrix determinants), Tusnady 2-box discrepancy (axis-aligned box deviation on point clouds), and Beck-Fiala signed-support L_inf (incidence rows) — none uses Matousek det-LB on the submatrix-determinant family of a sign matrix. **Field B** (complexity object): Sign-matrix rigidity R_M(r) for M in {+1,-1}^{N x N} in the Valiant 1977 / Friedman 1993 / Razborov 1989 / Lokam 2009 regime, accessed through the Friedman spectral lower bound F_r(M) := sigma_r(M)^2 <= R_M(r-1). Target: unsaturated rigidity bounds for explicit sign families (Hadamard, padded-identity, low-rank-plus-noise) — the canonical stepping stone toward log-depth arithmetic-circuit lower bounds for linear maps via the Valiant rigidity program.

Statement:

> For every sign matrix M in {+1,-1}^{N x N} with N = 2^n (n in {3, 4, 5}) and every r in {2, 3, 4}, define D_r(M) := max{|det(A)|^{2/r} : A is an r-by-r submatrix of M}, estimated by Monte Carlo sampling 2000 independent uniform r-subset row-by-column pairs and taking the maximum, and define F_r(M) := sigma_r(M)^2 (the r-th squared singular value of M, computed by exact SVD). Then 4 * F_r(M) >= D_r(M); equivalently sigma_r(M)^2 >= (1/4) * max{|det(A)|^{2/r} : A is an r-by-r submatrix of M}. A single (M, r, seed) triple producing a sampled r-by-r submatrix A with 4 * sigma_r(M)^2 < |det(A)|^{2/r} falsifies the conjecture.

Rationale (proposer's reasoning):

The compound-matrix identity sigma_1(M) * sigma_2(M) * … * sigma_r(M) = ||C_r(M)||_op gives a product upper bound on max{|det(A)|: A is r-by-r submatrix}, which by itself only yields a sigma_1^2-style ceiling. For pure-sign matrices the Marchenko-Pastur / Cauchy-interlacing concentration of singular values around sqrt(N) heuristically forces sigma_r within a bounded factor of sigma_1 (outside genuinely low-rank pathologies, where D_r also collapses), so the multiplicative compound bound should collapse to a per-singular-value bound sigma_r^2 >= D_r / 4 — transferring Matousek's combinatorial det-LB hereditary discrepancy directly into a Friedman spectral rigidity lower bound, a bridge absent from the literature.

Taxonomy category: `DISPERSION_DISCREPANCY` (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `c79380cd8d79f87f`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across 30 seeds x 3 sizes N in {8,16,32} x 5 ensembles x 3 ranks r in {2,3,4} (1350 trials), compute ratio = 4*sigma_r(M)^2 / D_r(M). Conjecture is SUPPORTED iff min ratio >= 1.0 over all 1350 trials AND mean ratio >= 1.5; FALSIFIED if any single trial yields ratio < 1.0.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.95	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.92	SAFE	SAFE
ALGEBRIZATION	SAFE	0.90	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: `NOVEL` against 11 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): `- Matousek determinant lower bound hereditary discrepancy matrix rigidity - submatrix determinant singular value sign matrix rigidity Friedman - det-LB discrepancy Hadamard matrix Valiant rigidity spectral lower bound`

Top relevant hits considered: - [http://arxiv.org/abs/2303.08167v2] On the Gap between Hereditary Discrepancy and the Determinant Lower Bound - [http://arxiv.org/abs/1101.0767v2] The determinant bound for discrepancy is almost tight - [http://arxiv.org/abs/2108.07945v2] A Tighter Relation Between Hereditary Discrepancy and Determinant Lower Bound - [http://arxiv.org/abs/2207.14810v3] Simplifying a classical-quantum algorithm interpolation with quantum singular value transformations - [http://arxiv.org/abs/math/0407301v1] Singular chain intersection homology for traditional and super-perversities - [http://arxiv.org/abs/2004.09833v2] Expressing the largest eigenvalue of a singular beta F-matrix with heterogeneous hypergeometric functions - [http://arxiv.org/abs/0909.2030v2] Size Bounds for Conjunctive Queries with General Functional Dependencies - [http://arxiv.org/abs/2103.05631v2] Improved upper bounds for the rigidity of Kronecker products

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def gaussian_elimination(A):
        n = len(A)
        for i in range(n):
            max_row = max(range(i, n), key=lambda r: abs(A[r][i]))
            A[i], A[max_row] = A[max_row], A[i]
            factor = 1 / A[i][i]
            for j in range(n):
                A[i][j] *= factor
            for k in range(n):
                if k != i:
                    factor = A[k][i]
                    for j in range(n):
                        A[k][j] -= factor * A[i][j]
        return A

    def matrix_multiply(A, B):
        n = len(A)
        C = [[0] * n for _ in range(n)]
        for i in range(n):
            for j in range(n):
                for k in range(n):
                    C[i][j] += A[i][k] * B[k][j]
        return C

    def svd(A):
        U, S, Vt = [], [], []
        n = len(A)
        Q, R = gaussian_elimination(A), [[A[j][i] for j in range(n)] for i in range(n)]
        for i in range(n):
            u_i = [R[i][j] / math.sqrt(sum(R[j][k]**2 for k in range(n))) for j in range(n)]
            U.append(u_i)
            s_i = sum(u_i[k] * R[k][i] for k in range(n))
            S.append(s_i)
            v_i = [R[j][i] / s_i if abs(s_i) > 1e-9 else 0 for j in range(n)]
            Vt.append(v_i)
        return U, S, Vt

    def det(A):
        n = len(A)
        if n == 1:
            return A[0][0]
        det_val = 0
        sign = 1
```

```
    for i in range(n):
        submatrix = [row[:i] + row[i+1:] for row in A[1:]]
        det_val += sign * A[0][i] * det(submatrix)
        sign *= -1
    return det_val

def random_subsets(rows, cols, r):
    rows_set = set(random.sample(range(len(rows)), r))
    cols_set = set(random.sample(range(len(cols)), r))
    submatrix = [[rows[i][j] for j in cols_set] for i in rows_set]
    return submatrix

def max_det_power(A, r):
    n = len(A)
    max_det = 0
    for _ in range(2000):
        submatrix = random_subsets(A, A[0], r)
        det_val = abs(det(submatrix))
        if det_val > max_det:
            max_det = det_val
    return max_det**(2/r)

def frobenius_norm(A):
    n = len(A)
    norm = 0
    for i in range(n):
        for j in range(n):
            norm += A[i][j]**2
    return math.sqrt(norm)

def run_trial_inner(N, r):
    M = [[random.choice([-1, 1]) for _ in range(N)] for _ in range(N)]
    U, S, Vt = svd(M)
    sigma_r = S[r-1]
    D_r = max_det_power(M, r)
    return {"sigma_r": sigma_r, "D_r": D_r}

n_values = [8, 16, 32]
r_values = [2, 3, 4]
results = []

for N in n_values:
    for r in r_values:
        result = run_trial_inner(N, r)
        sigma_r = result["sigma_r"]
        D_r = result["D_r"]
        ratio = 4 * sigma_r**2 / D_r
        results.append({"N": N, "r": r, "ratio": ratio})

min_ratio = min(result["ratio"] for result in results)
mean_ratio = sum(result["ratio"] for result in results) / len(results)

return {
    "metric_name": "4 * sigma_r^2 / D_r",
    "metric_value": mean_ratio,
    "instances_tested": len(results),
```

```
        "conjecture_holds": min_ratio >= 1.0 and mean_ratio >= 1.5,
        "counterexample": "" if min_ratio >= 1.0 else f"min_ratio={min_ratio}"
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i + 1 for i in range(3, 6)]

    for seed in seeds:
        trial = run_trial(seed)
        print(f"TRIAL: {trial}")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_419a13e3.py", line 129, in <module>
    trial = run_trial(seed)
            ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_419a13e3.py", line 107, in run_trial
    result = run_trial_inner(N, r)
             ^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_419a13e3.py", line 96, in run_trial_inn
    U, S, Vt = svd(M)
               ^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_419a13e3.py", line 48, in svd
    Q, R = gaussian_elimination(A), [[A[j][i] for j in range(n)] for i in range(n)]
           ^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_419a13e3.py", line 26, in gaussian_elim
    factor = 1 / A[i][i]
             ~~^~~~~~~~~
ZeroDivisionError: float division by zero
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test crashed with a ZeroDivisionError inside a hand-rolled Gaussian elimination SVD routine before any of the 1350 trials produced data, so neither the support nor the falsification criterion can be evaluated. | next: Replace the custom gaussian_elimination/svd implementation with numpy.linalg.svd (or add partial pivoting) and rerun the full 30-seed x 3-size x 5-ensemble x 3-rank sweep.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	399018
2	preregistration	claude_max	opus	0	0	5854
3	novelty	claude_max	opus	0	0	3365
4	novelty	claude_max	opus	0	0	8707
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17618
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	20617
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18598
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12651
9	judge	claude_max	opus	0	0	4103

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 490530 ms total latency. Provider mix: {'claude_max': 5, 'ollama_remote': 4}

(full prompt+response transcripts available in `research/audit/2c4a5e3ab436.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/2c4a5e3ab436.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/2c4a5e3ab436.tar.gz` (if generated)